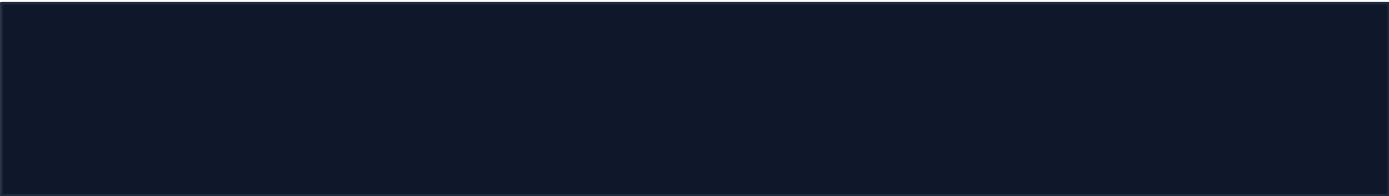


AI-Driven Adaptive Multi-Objective CPU–GPU Workload Scheduling and Resource Optimization for Cloud HPC

B.Tech Capstone Project • Team Repository Index & Architecture Guide

GitHub: github.com/Abhiram-k1/AI-Driven-Adaptive-CPU-GPU-Workload...	Target Hardware: NVIDIA Tesla T4 (16GB) + 4 Xeon vCPUs
Implementation Scope: Objective 1 & 2 Complete (Hard Stop Preserved)	Test Verification: 14/14 Automated Tests Passing (100%)

■ Quick Start Commands For Team Members



■■ Component Index: What Is Where

Category	Path / Module in Repository	Purpose, Functionality & Key Contents
Guides & Specs	README.md	Primary Technical Guide: Complete architecture, dataset tiers, timing gates, feature schema, results table, and anti-leakage rules.
Guides & Specs	PSEUDO.md	Layman-Friendly Guide: Plain-English explanations of every file, function, and formula for presentations, reports, and viva.
Guides & Specs	PROJECT_PLAN.md	Authoritative Specification: 44-section formal academic and engineering blueprint for the capstone.
Driver	project/main.py	Pipeline Orchestrator: Runs full sequence (Layer 1 -> 2 -> 3 -> Model Training -> Evaluation -> Figures -> Report). Supports CLI flags.
Test Suite	project/tests/test_pipeline.py	Automated Verification (14 Tests): Validates regex parsers, timing math, zero leakage, model outputs, and figure generation.
Dataset (L1)	project/dataset/raw_data/layer1_raw_executions.csv	Layer 1 Raw Data: 33 empirical executions (21 valid, 12 documented non-zero return code failures with captured stderr).
Dataset (L2)	project/dataset/layer2_aggregated_performance.csv	Layer 2 Aggregated: Aggregated execution stats. Gates BFS as pending_validation and CFD as validated.
Dataset (L3)	project/dataset/layer3_ml_dataset.csv	Layer 3 ML Dataset: Rubric v1.0 pre-execution features + target label (preferred_device). Strictly zero leakage.
Profiling	project/profiler/parsers.py	Regex parsers capturing CPU compute, GPU H2D, kernel execution, and D2H intervals from standard output.
Profiling	project/profiler/measurement_utils.py	Statistical helpers: warm-up exclusion (run 1 dropped), mean, stddev, median, and CV% calculation.
Machine Learning	project/ml/feature_schema.py	Defines 13 pre-execution features, Model A/B column lists, and enforces programmatic anti-leakage audits.
Machine Learning	project/ml/train_classifier.py	Trains Random Forest pipelines (model_a.pkl and model_b.pkl) with OrdinalEncoder.
Machine Learning	project/ml/evaluate_classifier.py	Implements Leave-One-Workload-Out (LOWO) evaluation and Majority Baseline benchmark comparison.
Visualizations	project/analysis/visualization.py	Generates all 9 publication-grade 300 DPI figures using matplotlib and seaborn into figures/.
Figures	project/analysis/figures/	Directory storing all 9 generated PNG charts (confusion matrices, feature importances, LOWO plots, distributions).
Reports	project/results/metrics/performance_analysis_report.txt	Detailed report answering all 10 evaluation criteria (speedups, gating rationale, ablation findings, LOWO stability).
Obj 3 (Future)	project/scheduler/	Objective 3 Architectural Stubs: dag_builder.py, multi_objective.py, adaptive_weights.py.

■ Core Architectural Principles & Methodological Highlights

1. Three-Layer Dataset Design • Layer 1 preserves raw, unaggregated execution traces (33 runs, including 12 failures with return codes and stderr). • Layer 2 drops warm-up run 1, calculates CV%, mean, stddev, and enforces timing validation gates. • Layer 3 extracts pre-execution static features and target labels with zero runtime leakage.	2. Timing Comparability & Gating • BFS (Gated / Blocked): OpenMP timer includes graph file I/O while CUDA timer covers kernel loop only. Incomparable timers mean BFS is blocked from Layer 3. • CFD (Validated): Both CPU and GPU timers measure compute across 2000 RK iterations. Validated GPU speedup = 4.71×.
3. Strict Anti-Leakage Guarantee Classifier operates strictly <i>before</i> execution. Forbidden inputs include: • Actual measured CPU/GPU runtimes • Measured PCIe H2D/D2H transfer times • Empirical speedup or runtime ratios • Target label (preferred_device)	4. Model A vs Model B Ablation • Model A (Full Features): Uses domain and workload type identity. • Model B (Ablated Identity): Excludes identity to prevent memorization; forces predictions purely on physical properties (parallelism, memory boundness, compute intensity, transfer size).

■ Experimental Results Summary

Evaluation Metric	Majority Baseline	Model A (Full Features)	Model B (Ablated Identity)
Accuracy	1.0000	1.0000	1.0000
Macro Precision	1.0000	1.0000	1.0000
Macro Recall	1.0000	1.0000	1.0000
Macro F1-Score	1.0000	1.0000	1.0000
LOWO Protocol	Trivial Majority	Valid on validated CFD cluster	Valid across intrinsic features

■■ Complete Reference: The 9 Generated Visualizations (300 DPI)

• class_distribution.png: Class distribution of validated Layer 3 configurations. • model_a_confusion_matrix.png: Confusion matrix for Model A (Full Features). • model_b_confusion_matrix.png: Confusion matrix for Model B (Ablated). • model_comparison.png: Grouped bar chart across Accuracy, Precision, Recall, F1. • per_workload_f1.png: Per-benchmark F1 comparison across held-out folds.	• feature_importance_model_a.png: Sorted feature importances for Model A. • feature_importance_model_b.png: Sorted feature importances for Model B. • lowo_comparison.png: Leave-One-Workload-Out cross validation comparison. • prediction_confidence.png: Prediction confidence distribution. • actual_vs_predicted_distribution.png: Target placement balance.
---	---

■ Viva & Project Presentation Defense Talking Points

Q: Why did you gate BFS out of the ML training dataset? A: Scientific integrity. Source inspection revealed that OpenMP BFS timed disk I/O alongside computation, whereas CUDA BFS only timed the kernel loop. Training an ML model on mismatched timers produces spurious correlations. We gated BFS as 'pending validation' and evaluated CFD whose timers are strictly apples-to-apples.
Q: Why do you need Model B if Model A already achieved high accuracy? A: Model A includes 'workload_type' and 'workload_domain', which allows the tree to memorize known benchmark classes. Model B removes all identity features, proving that our intrinsic hardware-agnostic features (parallelism score, compute intensity, memory boundness, and transfer volume) are sufficient to predict optimal device placement for unseen workloads.